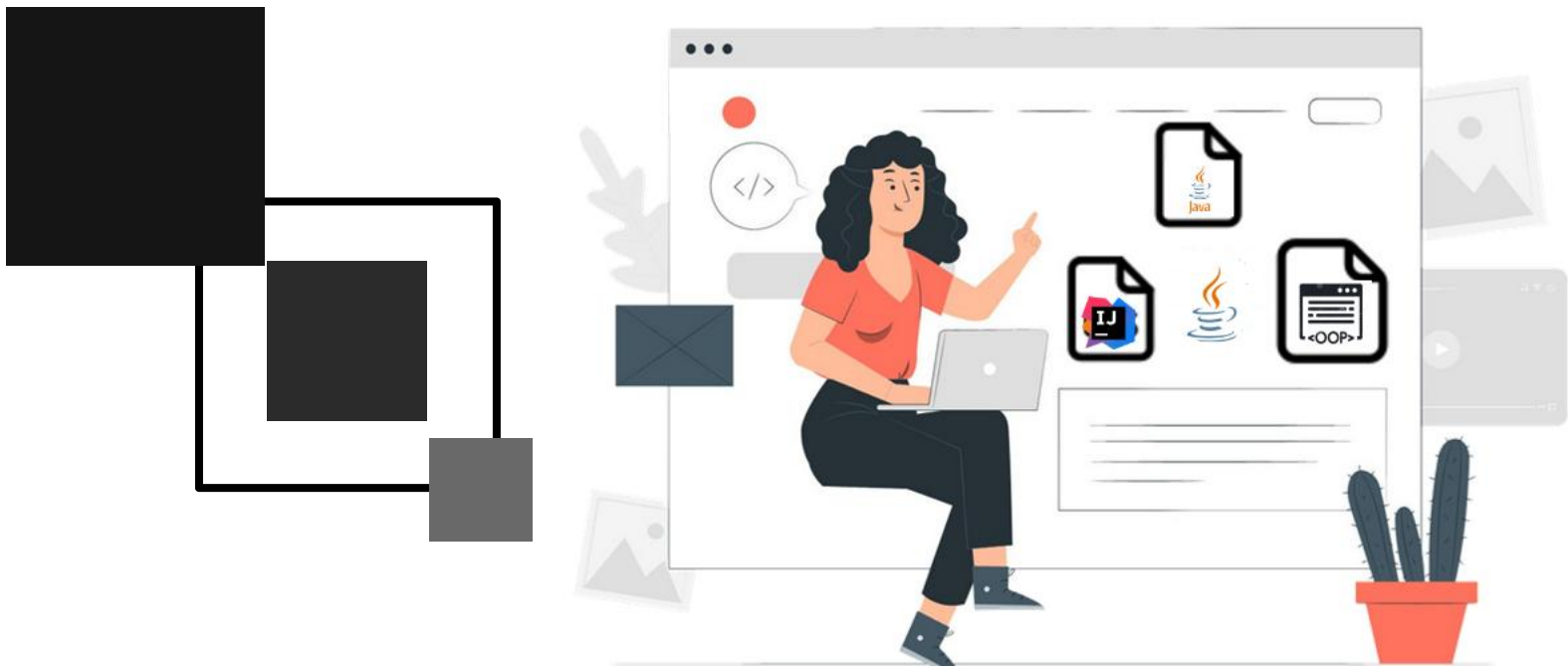




Domain Driven Design

Eliane Marion

FIAP



01

Annotation

Annotation

- As anotações na linguagem de programação Java servem para apoiar o compilador na execução do código fonte, começa sempre com @ e depois o nome, conforme iremos estudar. Dependendo da categoria da annotation, pode ser necessário incluir dados a ela, no formato de pares nome=valor. São três as categorias de anotações:

- *Anotações completas* – são aquelas que possuem múltiplos membros. Portanto, neste tipo, devemos usar a sintaxe completa para cada par nome=valor. Neste caso, cada par é informado separado do outro por uma vírgula. Por exemplo, `@Version(major=1, minor=0, micro=0)` é um caso de anotação completa.

Tipos anotação padrão

Os tipos anotação padrão são aquelas providas como parte do Java no pacote **java.lang**. Essas anotações podem ser utilizadas sem qualquer esforço adicional em suas aplicações, pois, visto que são parte de **java.lang**, nem mesmo precisam ser importadas.

@Override

É uma anotação marcadora que deve ser usada apenas com métodos. Serve para indicar que o método anotado está sobrescrevendo um método da superclasse.

- **Anotações marcadoras** – Não possui um membro. São identificadas apenas pelo nome, sem dados adicionais. Por exemplo, @Test na Listagem 2 é uma anotação marcadora;
- **Anotações de valor único** – São similares a anterior, porém, possuem um único membro, chamado valor. Elas são representadas pelo nome da anotação e um par nome=valor, ou simplesmente com o valor, entre parênteses. Em outras palavras, quando a anotação possui um único membro, só é necessário informar o valor, além do nome da anotação. Por exemplo, @MinhaAnotacao("valor") é um exemplo de sintaxe deste tipo de anotação;

Tipos anotação padrão

@Deprecated

Assim como @Override, é também uma anotação marcadora. Esta anotação é utilizada quando é necessário indicar que um método não deveria mais ser usado, ou seja, informa que o método está obsoleto. Diferente de @Override, @Deprecated deve ser colocada na assinatura do método.

```
@Deprecated public void getSalarioTotal () {  
    //return this.salario + bonus;  
    System.out.print(salario + "\n");  
}
```

Tag	Significado
@author	Identifica o autor da classe ou do método.
@deprecated	Identifica classes ou métodos obsoletos, e que não se recomenda seu uso porque possivelmente desaparecerá em posteriores. É interessante informar nessa tag, quais métodos ou classes podem ser usadas como alternativa ao método obsoleto.
@link	Possibilita a definição de um link para outro documento local ou remoto através de um URL.
@param	Mostra um parâmetro que será passado a um método.
@return	Mostra qual o tipo de retorno de um método. Não se deve poder usar em construtores ou métodos "void"

Tag	Significado
@see	Possibilita a definição referências de classes ou métodos, que podem ser consultadas para melhor compreender idéia daquilo que está sendo comentada.
@since	Indica desde quando uma classe ou métodos foi adicionado na aplicação.
@throws	Indica os tipos de exceções que podem ser lançadas pelo método.
@version	Informa a versão do método ou classe.

EXEMPLO DE COMENTÁRIO JAVADOC

```
1  /**Classe para objetos do tipo Funcionários, onde serão co
2  * @author Fulano
3  * @version 1.05
4  * @since Release 02 da aplicação
5  */
6  public class Funcionarios {...}
7
8  /** Método para retorno do salário do funcionário
9  *   @return Double - Valor do Salário */
10 public Double getSalario(){...}
11
12 /**Método para calculo da diária com base no salário do
13 * funcionário dividido pelo mês comercial de 30 dias para
14 * de cálculo de ajuda de custo para viagem.
15 * @author Ciclano
16 * @param diasViagem int - Valor total das vendas do mês.
17 * @param valorDeslocamento Double - Valor pago em cada di
18 * @return Double - Valor da diária
19 */
20 public Double calculaAjudaCusto(int diasViagem, Double va
21
22 /**Método para calculo do valor da bonificação baseada na
23 * seguinte faixa de valores: Para vendas menores de
24 * 25.000,00, o percentual de comissão aplicado será de 5%,
25 * será de 10%
26 * @author Beltrano
27 * @param valorVendas - Valor total das vendas do mês
28 * @return Double - Valor do resultado do cálculo conforme
29 * @throws IllegalArgumentException Se valorVendas é null.
30 */
31 public Double calculaBonificacao(Double valorVendas) throw
32
33 /**
34 * Tenta adicionar um cliente e retorna se esse cliente foi
35 * @param cliente o cliente a ser adicionado</em>
36 * @return se o cliente foi corretamente adicionado
37 */
38 public boolean adicionar(Cliente cliente){...}
```

Javadoc

É um utilitário fornecido pela Sun Microsystems junto ao JDK para gerar documentação de códigos Java em formato HTML a partir de comentários no código fonte Java. O Javadoc é suportado pela maioria dos IDEs de mercado.

É gerado a partir de um comentário no código fonte iniciado com “`/**`” e ter finalizado com “`*/`”. Dentro do comentário Javadoc é possível usar tags HTML e tags reservadas do Javadoc precedidas pelo caracter “`@`”, veja relação das tags reservadas mais utilizada na tabela 1.

Por convenção, sugere-se alocar os blocos de comentários, antes da definição de uma classe, interface, atributos, ou métodos, para introdução conceitual ao referido código.

Javadoc

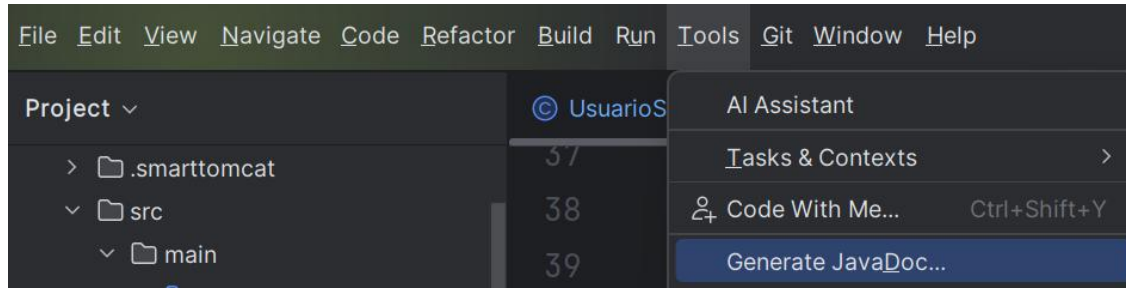
```
package br.com.fiap.models;

import jakarta.xml.bind.annotation.XmlRootElement;

/**
 * Essa classe foi criada para definição dos Usuários do sistema
 * e teste da criação do Javadoc
 * @author Eliane Marion
 * @version 1.0.0
 */
18 usages  👤 Eliane +1
@XmlRootElement
public class Usuario {
    3 usages
    private int id;
    4 usages
    private String nome;
    5 usages
```

Javadoc

Após fazer todos os comentários necessários você pode gerar um html com o javadoc completo ou por classe. Selecione a opção Generate JavaDoc no menu Tools, conforme imagem abaixo:



Javadoc

1. Output Directory

Define onde os arquivos HTML do Javadoc serão salvos. Por padrão, o IntelliJ sugere algo como:

`<seu-projeto>/out/javadoc`

2. Scope

Escolhe quais partes do código o Javadoc será gerado:

Project → Gera documentação de todo o projeto.

Module → Apenas do módulo atual.

Package → Apenas de um pacote específico.

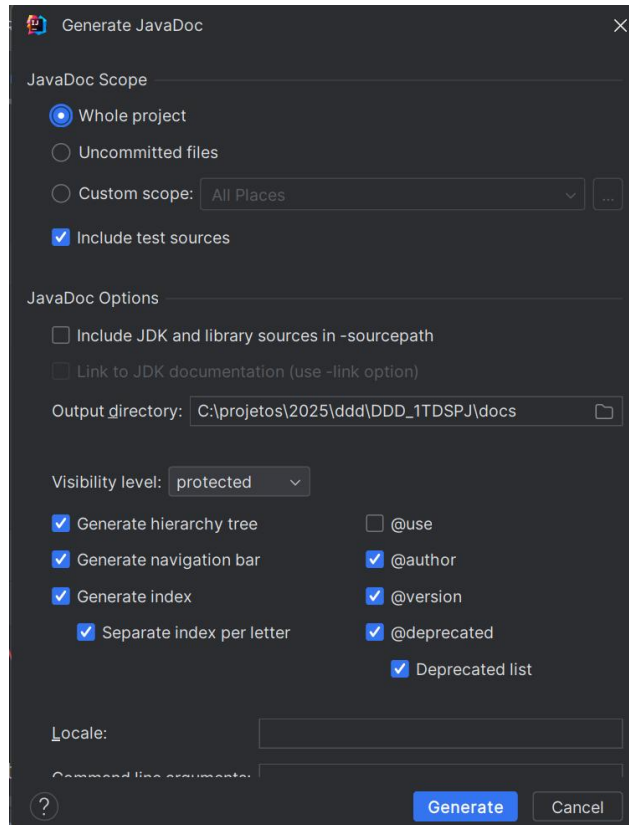
File → Apenas do arquivo atual aberto no editor.

3. Locale

Define o idioma da documentação gerada. Exemplo:

en_US → inglês (padrão)

pt_BR → português (se quiser que os rótulos do Javadoc apareçam traduzidos)



4. Visible Members

Controla quais membros (métodos, atributos etc.) serão incluídos:

Public → só membros públicos

Protected → inclui públicos e protegidos

Package → inclui também os de pacote (sem modificador)

Private → inclui tudo, até os privados

Geralmente, escolhe-se Public para documentação de API pública.

5. Other command line arguments

Permite passar parâmetros extras diretamente para o comando javadoc. Por exemplo:

-author → adiciona o autor nas páginas

-version → inclui a versão do arquivo

-link <https://docs.oracle.com/en/java/javase/17/docs/api/> → cria links para a documentação oficial da JDK

-encoding UTF-8 → garante que acentuação apareça corretamente

-windowtitle "Meu Projeto" → define o título da janela do navegador

Include test sources: Se marcada, inclui também o código de teste (src/test/java) na documentação. Normalmente desmarcada, pois testes não fazem parte da API pública.

Open generated documentation in browser: Abre automaticamente a documentação gerada no navegador padrão após o processo.

Generate hierarchy tree: Gera um gráfico hierárquico de classes e interfaces (por exemplo, mostrando quais classes herdam de qual).

Generate navigation bar: Adiciona menus de navegação no topo de cada página (pacotes, classes, índice).

Generate index: Cria um índice alfabético de todas as classes e membros documentados (muito útil para navegação rápida).

Generate deprecated list: Cria uma página separada listando todos os elementos marcados com @deprecated.

Include Include tag sections: Faz o javadoc incluir as seções de tags como @param, @return, @throws e @see nos HTMLs.

Include ‘Use’ page: Gera páginas “Use” mostrando onde cada classe ou método é utilizado dentro do projeto.

Command line preview: Mostra o comando completo javadoc que o IntelliJ vai executar, conforme suas opções selecionadas. Isso é ótimo se quiser entender o que o IntelliJ está fazendo “por baixo dos panos”

Resultado Final: Depois de gerar, você encontrará: /out/javadoc/index.html

Abra esse arquivo no navegador, é a página inicial da documentação.

 Generate JavaDoc ✕

JavaDoc Scope

☒ Whole project

☐ Uncommitted files

☐ Custom scope: All Places ...

☐ Include test sources

JavaDoc Options

☐ Include JDK and library sources in -sourcepath

☐ Link to JDK documentation (use -link option)

Output directory: C:\projetos\2025\ddd\DDD_1TDSPJ\docs 📁

Visibility level: public ▼

☒ Generate hierarchy tree

☒ @use

☒ Generate navigation bar

☒ @author

☒ Generate index

☒ @version

☒ Separate index per letter

☒ @deprecated

☒ Deprecated list

Locale: pt_BR

Command line arguments:

? Generate Cancel

← ↺ 🏠 Arquivo C:/projetos/2025/ddd/DDD_1TDSPJ/docs/index.html

OVERVIEW

PACKAGE

CLASS

USE

TREE

INDEX

HELP

Packages

Package	Description
br.com.fiap.dao	
br.com.fiap.dto	
br.com.fiap.models	
br.com.fiap.resource	
br.com.fiap.security	
br.com.fiap.service	